UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
ESCUELA DE CIENCIAS Y SISTEMAS
MANEJO E IMPLEMENTACIÓN DE ARCHIVOS
SECCIÓN A
PRIMER SEMESTRE 2026
AUX EDGAR JOSIAS CAN AJQUEJAY

MANUAL TÉCNICO

BRANDON EDUARDO PABLO GARCIA
202112092
Guatemala, marzo del 2026

Introducción

La aplicación desarrollada implementa una simulación del sistema de archivos EXT2/EXT3 mediante una arquitectura cliente-servidor, donde el frontend proporciona la interfaz gráfica para el usuario y el backend se encarga de la lógica del sistema de archivos, gestión de estructuras internas y ejecución de comandos.

El sistema permite a los usuarios crear y administrar discos virtuales representados como archivos binarios con extensión .mia los cuales contienen todas las estructuras de datos características de un sistema de archivos real: MBR (Master Boot Record), EBR (Extended Boot Record), Superbloque, Bitmaps de inodos y bloques, Tabla de Inodos y Bloques de Datos. Cada disco puede ser particionado, montado y formateado, permitiendo la creación de directorios y archivos con un esquema de permisos completo.

La interfaz gráfica, desarrollada en Angular con un diseño moderno y responsivo, ofrece dos áreas principales de interacción: una consola de comandos donde los usuarios pueden ejecutar scripts con todas las operaciones disponibles, y un explorador visual de solo lectura que permite navegar por la estructura de archivos y directorios de manera intuitiva, mostrando permisos, tamaños y contenido de los archivos.

Este manual técnico documenta de manera exhaustiva la arquitectura del sistema, el proceso de despliegue en AWS, las estructuras de datos fundamentales que componen el sistema de archivos, y cada uno de los comandos implementados, proporcionando una guía completa para comprender el funcionamiento interno de la aplicación y facilitar su mantenimiento y futura extensión.

Objetivos

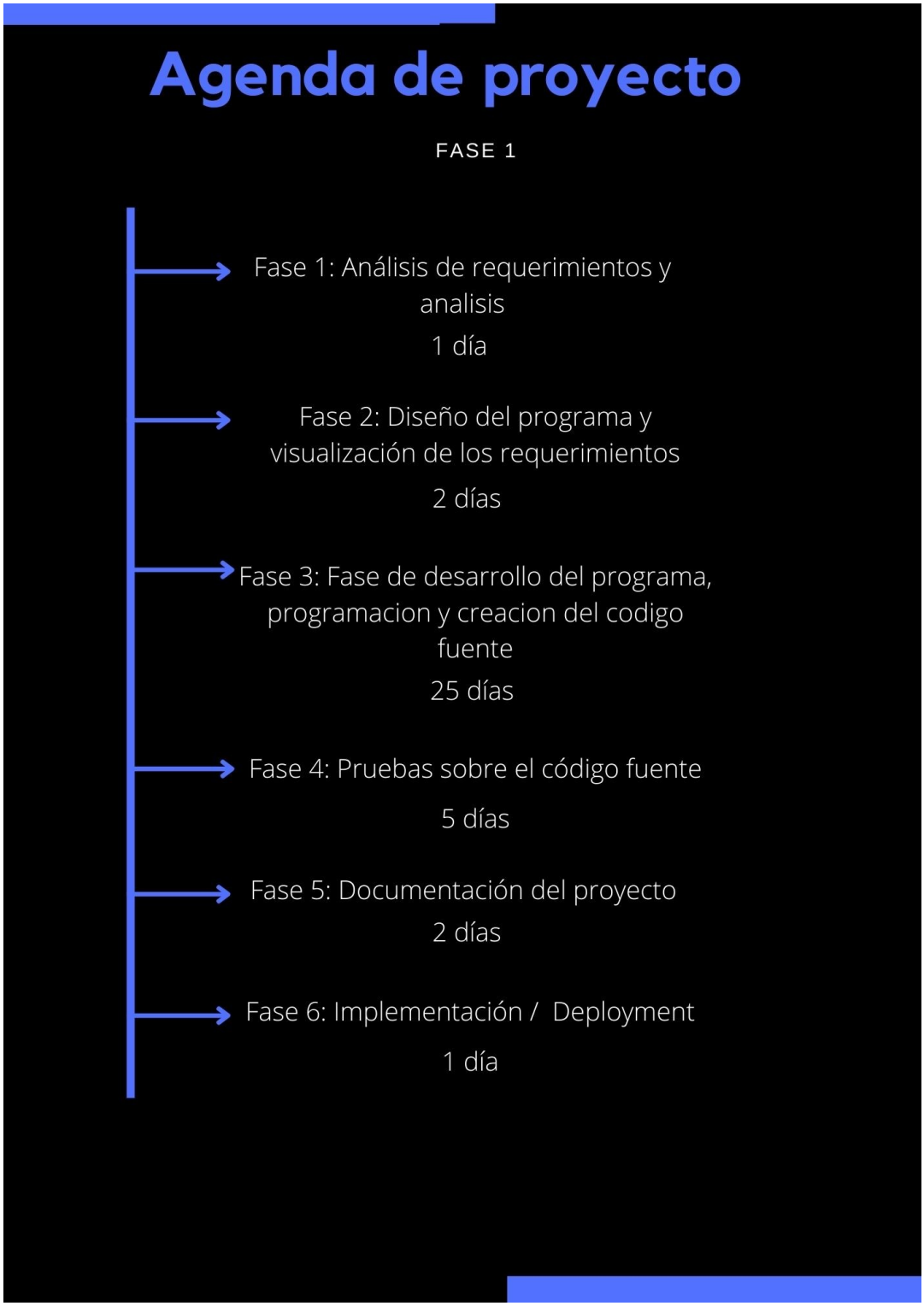
- Brindar la información necesaria para poder representar la funcionalidad técnica de la estructura, diseño y definición del aplicativo.

- Describir las herramientas utilizadas para el diseño y desarrollo del prototipo

Requerimientos de función

Requerimientos	Descripción
C++	Se recomienda el uso del g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0 o superior que fue la versión donde se programó el sistema de información
Visual Studio Code	Se recomienda el uso VSC como ID (versión 1.109.5 superior) que fue la versión donde se programó el sistema de información
Graphviz	Conocimiento sobre el uso de las librerías Graphviz para el uso de las representaciones gráficas
Angular	El frontend fue desarrollado utilizando el framework Angular, el cual permite construir una interfaz web moderna e interactiva.

Desarrollo

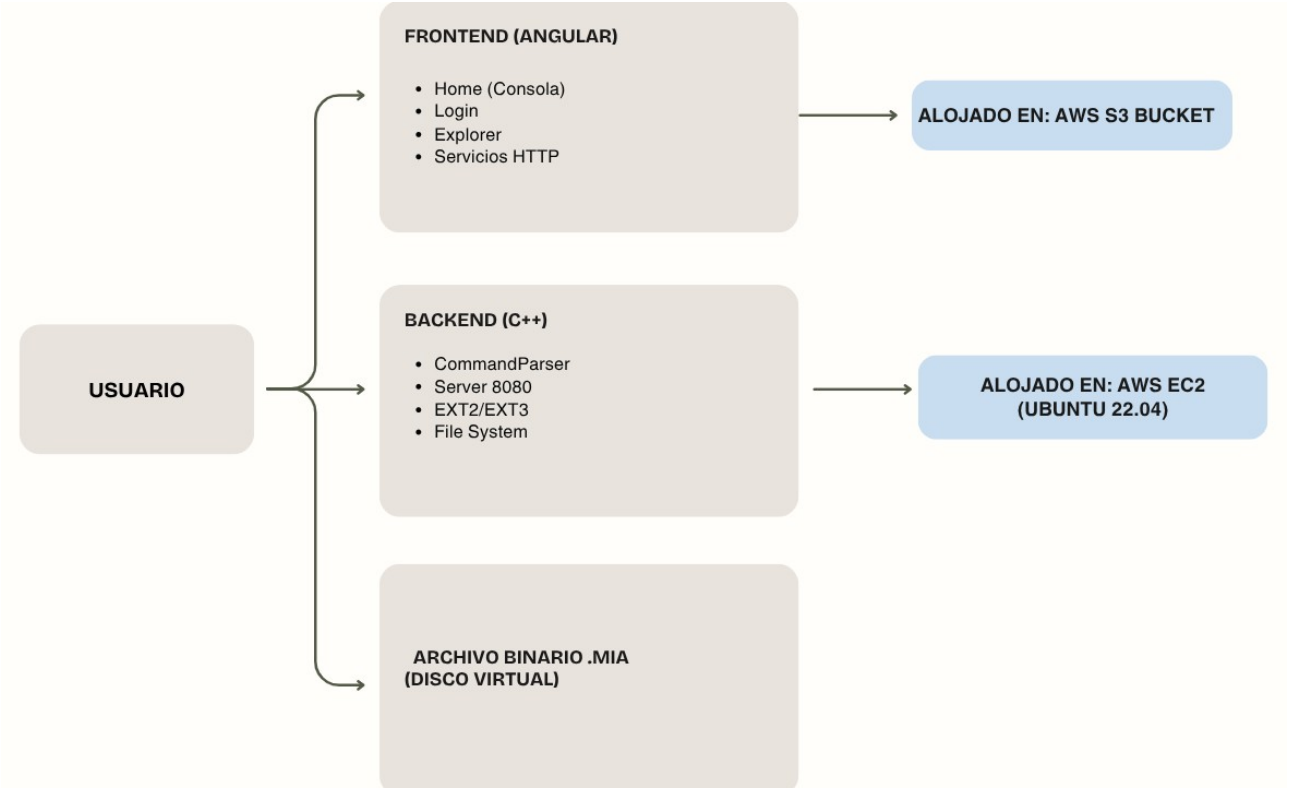

Agenda de proyecto
FASE 1
Fase 1: Análisis de requerimientos y analisis
1 día
Fase 2: Diseño del programa y visualización de los requerimientos
2 días
Fase 3: Fase de desarrollo del programa, programacion y creacion del codigo fuente
25 días
Fase 4: Pruebas sobre el código fuente
5 días
Fase 5: Documentación del proyecto
2 días
Fase 6: Implementación / Deployment
1 día

Contenido Técnico

Backend

Visión General

El sistema está compuesto por dos módulos principales que se comunican mediante HTTP/REST



Flujo de Comunicación

1. Usuario escribe comandos en el Home
2. Frontend envía POST /exec al backend
3. Backend parsea cada línea con CommandParser
4. Ejecuta cada comando sobre el archivo .mia
5. Devuelve respuesta como texto plano
6. Frontend muestra la respuesta en el panel de salida

Pasos del Despliegue

Backend (EC2)

- Crear instancia EC2:
- AMI: Ubuntu Server 22.04 LTS
- Tipo: t2.micro
- Grupo de seguridad: SSH (22), HTTP (80), HTTPS (443), TCP 8080

Frontend (S3)

- Crear bucket S3:

Nombre: mia-frontend-202112092
Acceso público habilitado

- Subir archivos:

Carpeta: dist/ext2-frontend/browser/
Todos los archivos al bucket

- Configurar hosting estático:

Índice: index.html
Error: index.html

- Política del bucket:

```
json
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicReadGetObject",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::mia-frontend-202112092/*"
  }]
}
```

El backend fue desarrollado en lenguaje C++ y es responsable de simular el comportamiento interno del sistema de archivos EXT2. Este módulo ejecuta los comandos recibidos desde el frontend y manipula directamente los archivos binarios que representan los discos virtuales.

Las principales responsabilidades del backend son:

- Interpretar y analizar los comandos ingresados por el usuario.
- Crear y administrar discos virtuales.
- Gestionar particiones dentro de los discos.
- Implementar las estructuras internas del sistema de archivos EXT2.
- Ejecutar operaciones sobre archivos y directorios.
- Generar reportes gráficos del sistema de archivos.

El backend funciona como un servidor HTTP que recibe solicitudes desde el frontend y devuelve los resultados de la ejecución de los comandos.

Comunicación entre Frontend y Backend

La comunicación entre ambos componentes se realiza a través de peticiones HTTP. Cuando el usuario ejecuta comandos desde la interfaz web, el frontend envía una solicitud al servidor backend con los comandos ingresados.

El backend procesa cada comando y devuelve una respuesta que contiene el resultado de la ejecución, la cual es mostrada en el área de salida del sistema dentro de la interfaz web.

Este modelo permite separar la lógica de procesamiento del sistema de archivos de la interfaz de usuario, facilitando el mantenimiento y escalabilidad del sistema.

```
1  import { Injectable, inject } from '@angular/core';
2  import { HttpClient } from '@angular/common/http';
3  import { Observable } from 'rxjs';
4
5  @Injectable({
6    providedIn: 'root'
7  })
8  export class BackendService {
9    private http = inject(HttpClient);
10   private apiUrl = 'http://localhost:8080';
11
12   ejecutarComandos(comandos: string): Observable<string> {
13     return this.http.post(`${this.apiUrl}/exec`, comandos, {
14       responseType: 'text'
15     });
16   }
17 }
```

Estructuras de Datos

Para simular el sistema de archivos EXT2 se implementaron diversas estructuras de datos que representan los componentes internos de un sistema de archivos real. Estas estructuras se almacenan dentro de archivos binarios con extensión .mia que representan discos virtuales.

MBR (Master Boot Record)

El MBR es la primera estructura almacenada en el disco virtual. Su función es almacenar la información principal del disco, incluyendo:

- Tamaño del disco
- Fecha de creación
- Identificador del disco
- Información de las particiones primarias

El MBR contiene un arreglo de particiones que permite registrar hasta cuatro particiones primarias o extendidas dentro del disco.

Una versión simplificada de la estructura utilizada en el sistema es la siguiente:

```
struct MBR {
    int mbr_tamano;
    time_t mbr_fecha_creacion;
    int mbr_disk_signature;
    Partition mbr_partitions[4];
};
```

mbr_tamano
Almacena el tamaño total del disco virtual en bytes. Este valor es utilizado para validar la creación de particiones dentro del disco.

mbr_fecha_creacion
Registra la fecha en la que fue creado el disco. Esto permite identificar cuándo fue generado el archivo binario que representa el disco virtual.

mbr_disk_signature
Es un identificador único del disco. Este valor ayuda a distinguir un disco de otro dentro del sistema.

mbr_partitions
Es un arreglo que almacena hasta cuatro particiones primarias o extendidas. Cada entrada contiene la información necesaria para localizar una partición dentro del disco. La estructura MBR se utiliza principalmente en el comando MKDISK, implementado en el archivo:

mkdisk.cpp

En este módulo se realiza la creación del archivo binario que representa el disco y posteriormente se escribe el MBR en la primera posición del archivo.

También es utilizada en:

fdisk.cpp

donde se actualiza la información de las particiones cuando se crean o eliminan.

EBR (Extended Boot Record)

El EBR se utiliza cuando existe una partición extendida. Dentro de la partición extendida se pueden crear múltiples particiones lógicas, las cuales son enlazadas mediante una lista de EBR.

Cada EBR almacena:

- Información de la partición lógica
- Tamaño de la partición
- Posición dentro del disco
- Apuntador al siguiente EBR

Esto permite recorrer todas las particiones lógicas dentro de una partición extendida.

```
struct EBR {
    char part_status;
    char part_fit;
    int part_start;
    int part_size;
    int part_next;
    char part_name[16];
};
```

part_status
Indica si la partición lógica se encuentra activa o eliminada.

part_fit
Define el tipo de ajuste utilizado para ubicar la partición dentro del espacio disponible.

part_start
Indica la posición dentro del disco donde comienza la partición lógica.

part_size
Almacena el tamaño de la partición lógica.

part_next
Es un apuntador al siguiente EBR dentro de la partición extendida. Esto permite crear una lista enlazada de particiones lógicas.

part_name
Nombre asignado a la partición lógica.

SuperBloque

El SuperBloque es una de las estructuras más importantes del sistema EXT2. Contiene información fundamental sobre el estado del sistema de archivos, como por ejemplo:

- Número total de inodos

- Número total de bloques
- Cantidad de inodos libres
- Cantidad de bloques libres
- Posición del bitmap de inodos
- Posición del bitmap de bloques
- Posición inicial de los inodos
- Posición inicial de los bloques

El SuperBloque permite al sistema administrar eficientemente el espacio disponible dentro del sistema de archivos.

```
struct Superblock {
    int s_filesystem_type;
    int s_inodes_count;
    int s_blocks_count;
    int s_free_blocks_count;
    int s_free_inodes_count;
    time_t s_mtime;
    time_t s_umtime;
    int s_inode_start;
    int s_block_start;
};
```

s_filesystem_type
Indica el tipo de sistema de archivos utilizado.

s_inodes_count
Número total de inodos dentro del sistema.

s_blocks_count
Número total de bloques de datos disponibles.

s_free_blocks_count
Cantidad de bloques libres disponibles.

s_free_inodes_count
Cantidad de inodos libres disponibles.

s_mtime
Fecha de la última vez que el sistema fue montado.

s_umtime
Fecha del último desmontaje del sistema.

s_inode_start

Posición dentro del disco donde comienza la tabla de inodos.

s_block_start
Posición dentro del disco donde comienzan los bloques de datos.

Inodos

Los inodos representan los metadatos de los archivos y directorios dentro del sistema de archivos. Cada archivo o carpeta tiene asociado un inodo.

Un inodo almacena información como:

- Identificador del usuario propietario
- Identificador del grupo
- Tamaño del archivo
- Permisos de acceso
- Fecha de creación
- Fecha de modificación
- Tipo de archivo (archivo o carpeta)
- Apuntadores a bloques de datos

Los inodos contienen múltiples apuntadores que permiten acceder a los bloques donde se almacena la información del archivo.

```
struct Inode {
    int i_uid;
    int i_gid;
    int i_size;
    time_t i_atime;
    time_t i_ctime;
    time_t i_mtime;
    int i_block[15];
    char i_type;
    int i_perm;
};
```

Bloques

Los bloques son las unidades básicas de almacenamiento de datos dentro del sistema de archivos. En esta implementación se utilizan bloques de tamaño fijo para almacenar el contenido de archivos o las entradas de directorios.

Existen diferentes tipos de bloques:

- Bloques de Carpeta
- Estos bloques almacenan las entradas de los directorios. Cada entrada contiene:

- Nombre del archivo o carpeta
- Número de inodo asociado

Esto permite organizar los archivos en forma de árbol jerárquico.

Bloques de Archivo

Estos bloques almacenan directamente el contenido de los archivos creados dentro del sistema.

Bloques de Apuntadores

Los bloques de apuntadores se utilizan cuando un archivo necesita más bloques de los que pueden ser referenciados directamente por el inodo. Estos bloques contienen referencias adicionales a otros bloques de datos.

Comandos Implementados

El sistema permite ejecutar diversos comandos que simulan operaciones sobre discos y sistemas de archivos.

MKDISK

Este comando permite crear un nuevo disco virtual.
Ejemplo:

mkdisk -size=50 -unit=M -fit=FF -path=/home/usuario/disco1.mia

Parámetros:
size: tamaño del disco
unit: unidad de medida (K o M)
fit: tipo de ajuste para las particiones
path: ruta donde se almacenará el disco

Este comando crea un archivo binario que representa el disco y escribe el MBR inicial.

RMDISK

Este comando elimina un disco virtual previamente creado.

Ejemplo:

rmdisk -path=/home/usuario/disco1.mia

El comando elimina el archivo binario correspondiente al disco.

FDISK

Permite crear, eliminar o modificar particiones dentro de un disco.

Ejemplo:

fdisk -size=10 -path=/home/usuario/disco1.mia -name=Part1

Este comando actualiza la estructura del MBR o EBR dependiendo del tipo de partición creada.

MOUNT

Permite montar una partición para que pueda ser utilizada por el sistema.

Ejemplo:

mount -path=/home/usuario/disco1.mia -name=Part1

Cuando una partición es montada se le asigna un identificador único que será utilizado por otros comandos.

MKFS

Este comando formatea una partición montada y crea las estructuras necesarias del sistema de archivos EXT2.

Ejemplo:

mkfs -id=921A -type=full

Este proceso crea el SuperBloque, los bitmaps, los inodos y los bloques iniciales del sistema de archivos.

LOGIN

Permite iniciar sesión dentro del sistema de archivos utilizando un usuario registrado.

Ejemplo:

login -user=root -pass=123 -id=921A

Este comando valida las credenciales dentro del archivo users.txt.

MKGRP

Permite crear un nuevo grupo de usuarios dentro del sistema.

Ejemplo:

mkgrp -name=grupo1

RMGRP

Elimina un grupo de usuarios existente.

MKUSR

Permite crear un nuevo usuario dentro de un grupo.

Ejemplo:

mkusr -user=usuario1 -pass=123 -grp=grupo1

RMUSR

Elimina un usuario existente.

MKDIR

Permite crear directorios dentro del sistema de archivos.

Ejemplo:

mkdir -path=/home/docs

MKFILE

Permite crear archivos dentro del sistema de archivos.

Ejemplo:

mkfile -path=/home/docs/archivo.txt -size=100

CAT

Permite mostrar el contenido de uno o varios archivos.

Ejemplo:

cat -file=/home/docs/archivo.txt

REP

Permite generar reportes gráficos de las estructuras internas del sistema de archivos.

Ejemplo:

rep -name=tree -path=/home/usuario/reporte.png -id=921A

Los reportes disponibles incluyen:

MBR

Disk

Inode

Block

Tree

SB

LS

File

Estos reportes permiten visualizar gráficamente la estructura interna del sistema de archivos.,

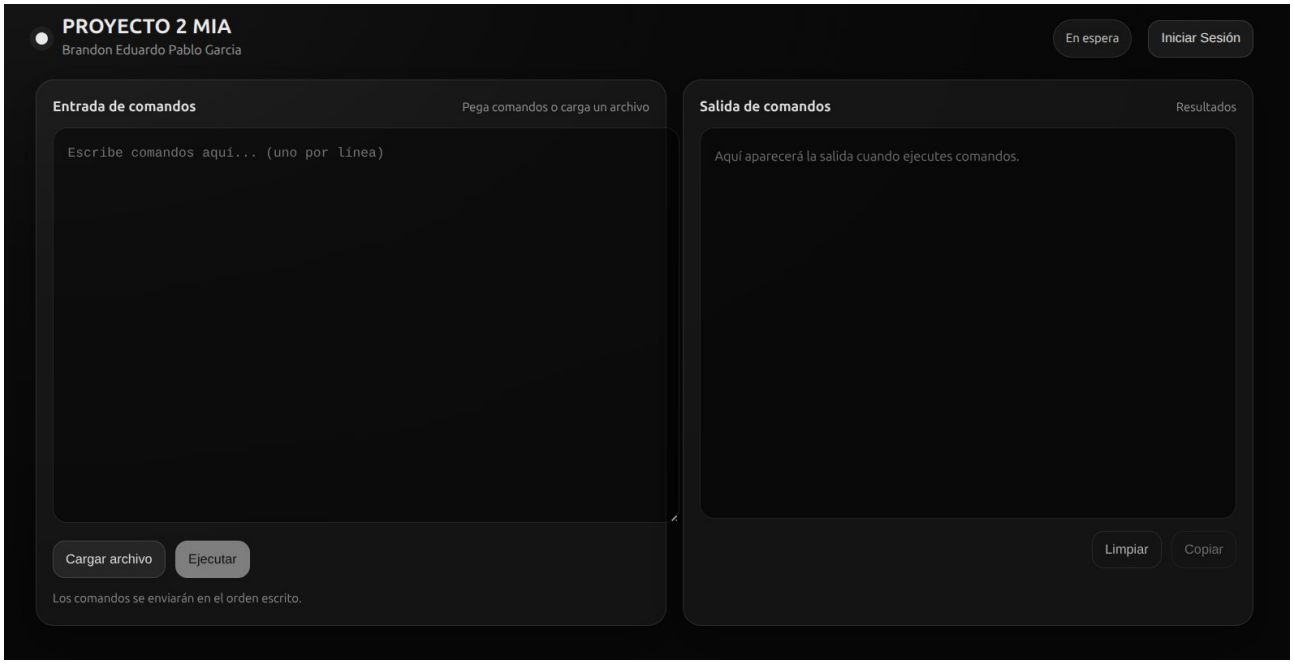
Frontend

El frontend fue desarrollado utilizando el framework Angular, el cual permite construir una interfaz web moderna e interactiva. Este componente se encarga de:

- Proporcionar una interfaz gráfica para que el usuario pueda ingresar comandos.
- Permitir la carga masiva de comandos mediante archivos de texto.
- Mostrar la salida generada por el backend.
- Visualizar los resultados de las operaciones realizadas sobre el sistema de archivos.

La comunicación con el backend se realiza mediante solicitudes HTTP. Cuando el usuario ejecuta comandos desde la interfaz, estos son enviados al servidor para su procesamiento.

El frontend incluye los siguientes elementos principales:

- Área de entrada de comandos
- Área de salida del sistema
- Botones de ejecución de comandos
- Carga masiva de archivos de comandos

- Interfaz visual para mejorar la experiencia del usuario